

TP2 Programmation avancé (Java)

Problème : Application de gestion de comptes

Nous voulons développer une application de gestion de compte pour une nouvelle banque qui prévoit un démarrage d'activité en 2026. Pour ce faire, les concepteurs ont jugé nécessaire la séparation des opérations en deux interfaces :

OperationSimple ayant les méthodes abstraites suivantes :

- **void crediter(int somme)**
- **boolean debiter(int somme)**

et **OperationAvancee** avec la méthode abstraite suivante :

- **boolean virement(Compte beneficiaire, int somme)**

A l'ouverture de ses portes, la banque envisage autoriser deux types de compte :

- compte courant et
- compte épargne.

Les deux comptes sont caractérisés par :

- un **identifiant** (de type int) et
- un **solde** (de type float).

Un compte courant aura de plus un attribut **decouvert** (de type float) et un compte épargne aura un **tauxInteret** (de type float). Dans tous les cas, un compte appartient à un seul client. La banque n'exclut pas la possibilité d'étendre son activité sur certains autres types de compte (compte conjoint, compte professionnel...).

Quel que soit le type de compte, nous devons permettre au titulaire de créditer ou débiter son compte. Cette dernière opération est possible si certaines conditions sont remplies :

- Pour un compte courant, on autorise l'opération si on ne dépasse pas la valeur de découvert après avoir débitée la somme souhaitée.
- Pour un compte épargne, on autorise l'opération si le solde ne devient pas négatif après avoir débitée la somme souhaitée.

Le virement n'est autorisé que depuis un compte courant. Le compte bénéficiaire peut être de type courant ou épargne. Dans tous les cas, on ne l'autorise que si on parvient à débiter la somme du compte émetteur.

Les méthodes **virement** et **debiter** retournent **true** en cas de réussite de l'opération, **false** en cas d'échec.

Pour chaque client, nous voulons enregistrer

- son **identifiant** (de type int),
- son **nom** (de type String) et
- son prénom (de type String).

Un client de la banque peut avoir un compte courant, jusqu'à trois maximum, ainsi qu'un compte épargne. Quel que soit le nombre de ses comptes, nous devons lui permettre de les consulter.

Questions :

1. Créez les classes **Compte**, **CompteCourant** et **CompteEpargne**. Générez / définissez les getters / setters et constructeurs.
2. Créez les interfaces **OperationSimple** et **OperationAvancee** et ajoutez les implémentations nécessaires.
3. Implémentez la méthode **debiter()** dans les deux classes **CompteCourant** et **CompteEpargne**.
4. Quel que soit le type de compte, la méthode **crediter()** a une seule implémentation et ne changera pas lorsque la banque décidera de s'ouvrir sur d'autres types de compte. Proposez la meilleure implémentation de cette méthode.
5. Implémentez la méthode **virement()**.
6. La classe **Compte** doit forcer **CompteCourant** et **CompteEpargne** à avoir leur propre implémentation de la méthode **void imprimer()** : cette méthode affiche toutes les caractéristiques d'un compte (tout sauf identifiant) ainsi que son type (courant ou épargne). Implémentez la méthode **imprimer** dans les deux classes **CompteCourant** et **CompteEpargne**.
7. Créez la classe **Client** et générez / définissez les getters / setters et constructeur.
8. Ajoutez les attributs nécessaires qui permettront de déterminer le nombre de comptes courants d'un client.
9. Dans **Client**, définissez une méthode **ajouterCompteCourant(CompteCourant compte)** qui permet d'ajouter un nouveau compte courant et qui retourne **true** en cas de réussite, **false** en cas d'échec.
10. Dans **Client**, définissez une méthode **ajouterCompteEpragne(CompteEpargne compte)** qui permet d'ajouter un compte épargne et qui retourne **true** en cas de réussite, **false** en cas d'échec.
11. Définissez une méthode **void consulter()** qui affiche les données du client ainsi que les détails de tous ses comptes.
12. Dans **CompteEpargne**, écrivez une méthode **ajouterInteret()** qui ajoute les intérêts au solde en fonction de la valeur de l'attribut **tauxInteret**.
13. Testez progressivement toutes les méthodes implémentées et les méthodes à implémenter dans **main**.
14. Faites les changements nécessaires pour permettre à un client d'effectuer un virement depuis son compte épargne vers un de ses comptes courants (bien sûr si débiter son compte épargne est possible).